Troubleshooting_Guide

WebSocket Monitoring System - Troubleshooting Guide

Common Issues and Solutions

🚨 Critical Issues

Issue: WebSocket Connection Failed

Symptoms:

- Dashboard shows "Disconnected" status
- No events appear in monitoring
- Error: `WebSocket connection failed`

Solutions:

1. **Check if monitoring server is running:**

   ```
   lsof -i :8090
   ```

2. **Start monitoring server:**

   ```
   go run ./cmd/monitor-server
   ```

3. **Verify correct port in configuration:**
   - Server must run on port 8090
   - Check `internal/working/config.distributed.json`

4. **Check firewall settings:**
   - Ensure port 8090 is not blocked
   - Test with: `telnet localhost 8090`

Issue: SSH Worker Connection Failed

Symptoms:

- Error: "Failed to connect to SSH worker"
- Demo falls back to local mode
- Status shows connection errors

Solutions:

1. **Test SSH connection manually:**

```
ssh -o ConnectTimeout=10 milosvasic@localhost 'echo "SSH
        works"'
```

2. **Check SSH credentials:**

```
echo "Host: $SSH_WORKER_HOST"
echo "User: $SSH_WORKER_USER"
echo "Port: $SSH_WORKER_PORT"
```

3. **Verify SSH service:**

```
# Check if SSH server is running
sudo systemctl status sshd

# Check if SSH port is open
netstat -tlnp | grep :22
```

4. **Configure SSH key authentication (recommended):**

```
# Generate SSH key if not exists
ssh-keygen -t rsa -b 4096

# Copy key to remote host
ssh-copy-id milosvasic@localhost

# Set environment variable
export SSH_PRIVATE_KEY_PATH=~/.ssh/id_rsa
```

Issue: LLM API Authentication Failed

Symptoms:

- Error: "API authentication failed"
- Translation stops immediately
- No progress updates

Solutions:

1. **Check API key:**

```
echo $OPENAI_API_KEY  # Should not be empty
echo $ANTHROPIC_API_KEY  # For Claude
echo $DEEPSEEK_API_KEY  # For DeepSeek
```

2. **Set API key:**

```
export OPENAI_API_KEY=your_actual_api_key_here

# Or add to ~/.bashrc for permanent
echo 'export OPENAI_API_KEY=your_key' >> ~/.bashrc
source ~/.bashrc
```

3. **Test API key:**

```
curl -H "Authorization: Bearer $OPENAI_API_KEY" \
     https://api.openai.com/v1/models
```

🔧 Configuration Issues

Issue: Port Already in Use

Symptoms:

- Error: "bind: address already in use"
- Server fails to start

Solutions:

1. **Find process using port:**

```
lsof -i :8090
```

2. **Kill the process:**

```
kill -9 <PID>

# Or find and kill by name
pkill -f "monitor-server"
```

3. **Change port in configuration:**

```
{
  "server": {
    "port": 8091  // Use different port
  }
}
```

Issue: File Not Found

Symptoms:

- Error: "no such file or directory"

- Demo fails to start

Solutions:

1. **Check required files:**

   ```
   # Test input file
   ls -la test/fixtures/ebooks/russian_sample.txt

   # Demo files
   ls -la demo-*-monitoring*.go

   # Config files
   ls -la internal/working/config.distributed.json
   ```

2. **Create missing files:**

   ```
   # Create test directory if missing
   mkdir -p test/fixtures/ebooks/

   # Create sample input file
   echo "Это образец русского текста." > test/fixtures/ebooks/
           russian_sample.txt
   ```

🌐 Network Issues

Issue: WebSocket Connection Timed Out

Symptoms:

- Connection attempts hang
- Dashboard shows loading state

Solutions:

1. **Check network connectivity:**

   ```
   # Test basic connectivity
   ping localhost

   # Test WebSocket endpoint
   curl -i -N -H "Connection: Upgrade" \
        -H "Upgrade: websocket" \
        -H "Sec-WebSocket-Key: SGVsbG8sIHdvcmxkIQ==" \
        -H "Sec-WebSocket-Version: 13" \
        http://localhost:8090/ws
   ```

2. **Check server logs:**

   ```
   tail -f monitor-server.log
   ```

3. **Increase timeout:**

```
// In client code
dialer := websocket.DefaultDialer
dialer.HandshakeTimeout = 10 * time.Second
```

🐛 Performance Issues

Issue: Slow Progress Updates

Symptoms:

- Progress bar updates slowly
- Large delays between events

Solutions:

1. **Check system resources:**

```
# CPU usage
top -p <monitor_server_pid>

# Memory usage
ps aux | grep monitor-server

# Network connections
netstat -an | grep :8090
```

2. **Reduce event frequency:**

```
// In translation code
if time.Since(lastUpdate) > 100*time.Millisecond {
    emitProgressEvent(...)
    lastUpdate = time.Now()
}
```

3. **Enable debug mode:**

```
export LOG_LEVEL=debug
go run ./cmd/monitor-server
```

📊 Dashboard Issues

Issue: Dashboard Not Loading

Symptoms:

- Browser shows blank page
- JavaScript errors in console
- Styles not applied

Solutions:

1. **Check browser console:**
   - Press F12 → Console tab
   - Look for JavaScript errors
   - Check network requests
2. **Verify file paths:**

```
# Check monitor.html exists
ls -la monitor.html

# Check file permissions
cat monitor.html | head -5
```

3. **Test with different browser:**
   - Chrome/Firefox/Safari
   - Clear browser cache
   - Disable extensions

Issue: Charts Not Displaying

Symptoms:

- Progress charts empty
- Chart.js errors
- Data not visualized

Solutions:

1. **Check Chart.js loading:**

```
<!-- Verify this line exists in HTML -->
<script src="https://cdn.tailwindcss.com"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

2. **Check browser network tab:**
   - Chart.js should load successfully
   - No 404 errors for CDN resources
3. **Test Chart.js manually:**

```
// In browser console
console.log(typeof Chart);
// Should output "function"
```

🔍 Debug Mode

Enable Comprehensive Logging

```
# Enable debug logging for server
export LOG_LEVEL=debug

# Enable debug for translation
export DEBUG=true

# Run with verbose output
go run ./cmd/monitor-server -v -log-level=debug
```

Monitor System Resources

```
# Monitor all system resources
htop

# Monitor network connections
watch -n 1 'netstat -an | grep :8090'

# Monitor WebSocket connections
watch -n 1 'lsof -i :8090'
```

Use Wireshark for Network Debugging

1. Start Wireshark
2. Filter: `tcp.port == 8090`
3. Start monitoring server
4. Connect client
5. Analyze WebSocket handshake

📋 Error Code Reference

Error Code	Description	Solution
`ECONNREFUSED`	Connection refused	Start monitoring server
`ETIMEDOUT`	Connection timeout	Check network/firewall
`EADDRINUSE`	Port already in use	Kill process or change port
`EACCES`	Permission denied	Check file permissions
`ENOTFOUND`	Host not found	Check DNS/host names
`EPIPE`	Broken pipe	Reconnect WebSocket
`ECONNRESET`	Connection reset	Check server stability

🛠️ Advanced Troubleshooting

WebSocket Protocol Debugging

```
# Use websocat for testing
brew install websocat
websocat ws://localhost:8090/ws

# Or use wscat
npm install -g wscat
wscat -c ws://localhost:8090/ws
```

SSL/TLS Issues (if using HTTPS)

```
# Check certificate
openssl s_client -connect localhost:8443

# Verify certificate chain
openssl verify server.crt
```

Memory Leak Detection

```
# Monitor memory usage over time
while true; do
    ps aux | grep monitor-server | grep -v grep
    sleep 5
done

# Or use pmap
pmap <monitor_server_pid>
```

📞 Getting Help

Collect Debug Information

```
# Create debug report
cat > debug-report.txt << EOF
=== WebSocket Monitoring Debug Report ===
Date: $(date)
System: $(uname -a)
Go Version: $(go version)

=== Running Processes ===
$(ps aux | grep -E "(monitor|translate)")

=== Network Status ===
$(netstat -an | grep -E "(8090|8080)")

=== Environment Variables ===
SSH_WORKER_HOST: $SSH_WORKER_HOST
```

```
SSH_WORKER_USER: $SSH_WORKER_USER
OPENAI_API_KEY: ${OPENAI_API_KEY:0:10}...

=== Recent Logs ===
$(tail -20 monitor-server.log 2>/dev/null || echo "No logs found")
EOF

echo "Debug report saved to debug-report.txt"
```

Create Minimal Reproduction

```
# Minimal WebSocket test
echo "package main

import (
    \"fmt\"
    \"time\"
    \"github.com/gorilla/websocket\"
)

func main() {
    conn, _, err := websocket.DefaultDialer.Dial(\"ws://
        localhost:8090/ws\", nil)
    if err != nil {
        fmt.Printf(\"Connection failed: %v\\n\", err)
        return
    }
    defer conn.Close()

    fmt.Println(\"Connected successfully!\")

    for i := 0; i < 10; i++ {
        err := conn.WriteJSON(map[string]interface{}{
            \"type\": \"test\",
            \"message\": fmt.Sprintf(\"Test message %d\", i),
        })
        if err != nil {
            fmt.Printf(\"Write error: %v\\n\", err)
            return
        }
        time.Sleep(1 * time.Second)
    }
}" > minimal-websocket-test.go

go run minimal-websocket-test.go
```

🔄 Recovery Procedures

Complete System Reset

```
#!/bin/bash
# Reset monitoring system

# 1. Kill all related processes
pkill -f "monitor-server"
pkill -f "demo-.*monitoring"

# 2. Clean up temporary files
rm -f *.log
rm -f demo_*_output.md

# 3. Reset ports
sudo lsof -ti:8090 | xargs -r sudo kill -9

# 4. Restart services
go run ./cmd/monitor-server &

echo "System reset completed"
```

Database/State Reset

```
# Clear session state
rm -rf /tmp/translate-ssh-*
rm -f .session-cache

# Reset WebSocket connections
curl -X POST http://localhost:8090/reset
```

Quick Reference Commands

Command	Purpose
`lsof -i :8090`	Check if monitoring server is running
`go run ./cmd/monitor-server`	Start monitoring server
`tail -f monitor-server.log`	Monitor server logs
`echo $OPENAI_API_KEY`	Check API key
`ssh milosvasic@localhost`	Test SSH connection
`curl http://localhost:8090/status`	Check server status

Remember: Most issues are caused by:

1. Server not running
2. Incorrect configuration
3. Network/firewall problems

4. Missing environment variables

Start with the simplest check and work through the solutions systematically!